

Cloud Computing – EC1 Assignment (20 Marks)

Telemetry Agent

Activity: Write a Bash shell script (**cloud_monitor.sh**) that acts as a lightweight, automated telemetry and log monitoring agent.

Goal: The script must do the following:

- Analyze local system metrics (like disk usage or CPU load)
- Parse a simulated web server access log to detect anomalies (e.g., HTTP 5xx errors).
- Compile these findings and output a structured JSON report, mimicking how local agents send data to a central cloud dashboard

Laying the groundwork:

- **Suggested Command Line Utilities:** Standard Unix utilities to extract data - awk, grep, sed, df, top
- **Log Simulation:** Create a dummy access.log file** in your workspace containing a mix of HTTP 200, 404, and 500 status codes to test your script.
- **The Output:** The final output printed to the terminal must be valid JSON.

Reason: This mimics one of the aspects of production readiness: a hypothetical centralized log server could easily ingest and parse the data.

- **Safety First (Guardrails):** Ensure that your script operates with the principle of least privilege.
 - Do not use commands that modify or delete system files (Exception: You may move/compress the dummy access.log file for the rotation task/Log archival add-on)
 - Use a read -p command to prompt the user with a "y/n" confirmation to begin the system scan, ensuring the agent doesn't consume CPU cycles without explicit permission.

Add-ons that need to be implemented: Once the basic metric-gathering loop starts working, add features (add-ons) to the agent.

1. **Webhook Alerting (Cloud Integration):** Give your agent the ability to "push" data. Use curl to send the final JSON payload via an HTTP POST request to a free webhook testing site (e.g., webhook.site) if any critical anomalies (like high error rates) are detected.
2. **Configuration File Abstraction:** Separate the core rules from the code. Have the script read a local text file (e.g., ~/.monitor_conf) that defines dynamic alert thresholds (e.g., MAX_DISK=80%, ERROR_LIMIT=5). **Reason:** This prevents hardcoding values and makes the agent portable across different lab instances.
3. **Audit Logging:** Accountability is a key aspect of DevOps or in fact for any project. Let the script (silently) append the run timestamp, the number of errors found, and the execution duration to a hidden history file (e.g., ~/.monitor_history).
4. **Log Archival (Rotation):** After the log file is successfully analyzed, automatically compress it using tar or gzip and move it to an ~/archive directory, appending the current date to the

filename (housekeeping activity). **Note:** You can also write a script to automatically generate a 1000+ lines access.log file with randomized IP addresses, timestamps, endpoints, and HTTP status codes. This will enhance your testing. When you write a script, make sure that you adhere to the **Format of the log file**. The script for your log file generation need not be submitted.

Grading Rubric (Total: 20 Marks)

How to Submit: To receive full credit, you must upload a document containing the **image snapshots** of your terminal and the complete code as a proof of your work.

Evaluation Criteria	Description	Required Proof (Image Snapshots)	Marks
1. Core: Data Extraction & JSON	Successfully extracts system metrics/log data and outputs a valid JSON summary.	• Screenshot of the script executing and displaying the correctly formatted JSON payload. • Explain your JSON data format • The complete code	8.0
2. Core: Guardrail Execution	Pauses to ask the user for "y/n" permission before initiating the heavy system scan.	Screenshot of the prompt pausing for input, typing "y", and the resulting execution.	2.0
Add-ons (#1 - #4)	Successful implementation of the add-on.	Screenshot clearly demonstrating the add-on (e.g., cat ~/.monitor_history, or the webhook site receiving the payload).	2.5 for each add-on

Screenshot - The screenshot must also show your shell/command prompt. Instead of the standard shell prompt (\$/#/...), **customize it to show your exact BITS id**.

Format of the access.log file:

```
127.0.0.1 - - [19/Aug/2026:10:00:01 +0530] "GET /index.html HTTP/1.1" 200 1024
127.0.0.1 - - [19/Aug/2026:10:05:12 +0530] "POST /api/data HTTP/1.1" 500 512
127.0.0.1 - - [19/Aug/2026:10:10:00 +0530] "GET /image.png HTTP/1.1" 404 256
127.0.0.1 - - [19/Aug/2026:10:12:00 +0530] "GET /api/login HTTP/1.1" 502 128
```

Example snippet of a customized prompt:

```
Last login: Mon Aug 17 15:14:13 2026
YourBITSID> ./cloud_monitor.sh
Proceed with system scan? (y/n):
```

Important Dates:

- **Announcement of the Assignment:** 23rd August 2026
- **Last date of Submission:** 7th October 2026 - cut off by 9 PM